

Impactis Project

Second Milestone Report — Review Improvements & Redis Caching Layer

1. Summary

This milestone covers two bodies of work: first, four improvements identified during the review of the first milestone (resource-state filtering, database indexing, code-quality tooling, and query-efficiency fixes); second, the addition of a Redis caching layer to the two most frequently called read endpoints, covering cache key design, TTL policy, cache invalidation, resilience when Redis is unavailable, and hit/miss observability. All work was verified with automated tests and manual evidence (live database inspection, query logs, and load testing), and the performance impact of caching was measured directly with k6.

2. First Milestone Review — Improvements

Four issues raised during the review of the first milestone were addressed prior to starting the caching work, so the caching layer would be built on a clean, corrected foundation.

2.1 Resource-State Filtering

The schema already defined **isActive** (restaurants), **isAvailable** (menu items), and **status** (users) columns, but no code path enforced them. This was corrected end-to-end:

- **GET /restaurants** now returns only restaurants where `isActive = true`, filtered at the database query level.
- **GET /menu?restaurantId=** now returns only menu items where `isAvailable = true`.
- **POST /order** now validates, before creating an order, that the user is **ACTIVE**, the restaurant is active, and every requested menu item is available — rejecting the request with a 409 Conflict and a clear message if not.

Verified with dedicated unit tests for each rejection path, plus updates to existing tests whose mock data previously omitted these fields.

2.2 Database Indexes

Explicit indexes were added on all foreign key columns used in frequent lookups: `menu_items.restaurant_id`, `orders.user_id`, `orders.restaurant_id`, `order_items.order_id`, and `order_items.menu_item_id`.

Verified by inspecting Postgres directly (ldi) after a full container rebuild — all five indexes confirmed present alongside primary-key and unique-constraint indexes.

2.3 Lombok Adoption

All 5 entities and 10 DTOs were refactored to use Lombok (`@Getter/@Setter` on entities, `@Data` on DTOs), removing several hundred lines of hand-written boilerplate. Entities intentionally avoid `@Data` itself, since generated `equals()/hashCode()/toString()` methods can trigger unwanted lazy-loading or unstable behaviour on JPA relationships.

Verified by a full clean rebuild and complete test suite pass after the refactor.

2.4 N+1 Query / Eager-Loading Fix

Every @ManyToOne relationship was made explicitly FetchType.LAZY (JPA's actual default for @ManyToOne is EAGER, unlike @OneToMany). This closes a silent performance gap where loading a restaurant, order, or menu item could have triggered unnecessary related-entity queries.

Verified with live evidence: Hibernate SQL logging was temporarily enabled and a GET /restaurants call against a restaurant with an attached menu item was inspected directly. Exactly one SQL SELECT was issued, with no follow-up queries against menu_items or orders — confirming no N+1 behaviour.

3. Second Milestone — Redis Caching Layer

A caching layer was added to the two most frequently called read endpoints, using Redis as the cache store and Spring's caching abstraction as the integration point.

3.1 Infrastructure

- Redis 7.2 (Alpine) added as a Docker Compose service alongside PostgreSQL, with a healthcheck and persistent volume.
- Spring Data Redis added for connectivity; connection host/port externalized via environment variables, consistent with the existing database configuration pattern.
- A dedicated RedisCacheManager configures per-cache serialization (JSON, via GenericJacksonJsonRedisSerializer) and per-cache time-to-live.

3.2 Cached Endpoints, Keys, and TTL

Endpoint	Cache Name	Key Pattern	TTL
GET /restaurants	restaurants	restaurants::all	10 minutes
GET /menu?restaurantId=	menu	menu::restaurant:{restaurantId}	5 minutes

The restaurant list changes infrequently and is given a longer TTL; menus change more often as items are added, so a shorter TTL keeps staleness bounded even without an explicit code path touching that cache.

3.3 Cache Miss Handling and Invalidation

- A cache miss falls through transparently to the existing repository query; the result is then written back to Redis by Spring's caching interceptor. No endpoint behaviour changes on a miss beyond the added latency of the database round-trip.
- Creating a restaurant evicts the restaurants::all cache entry, so the next list request reflects the new restaurant immediately rather than waiting out the TTL.
- Adding a menu item evicts only that restaurant's specific menu cache entry, leaving other restaurants' cached menus untouched.

Both invalidation paths were verified manually: a new restaurant was created and confirmed to appear in the very next list call; a new menu item was created and confirmed to appear in the very next menu call, in each case after first confirming the relevant Redis key had been removed.

3.4 Resilience When Redis Is Unavailable

Explicit connect and command timeouts (2 seconds) were configured, and a custom CacheErrorHandler was registered so that any Redis failure during a cache read, write, or eviction is logged as a warning and treated as a cache miss, rather than propagating as an exception.

Verified by stopping the Redis container and issuing a request to a cached endpoint: prior to this fix, the request hung and eventually returned a 500 error; after the fix, the same request returned a correct 200 OK response, sourced directly from PostgreSQL, with a warning logged noting the fallback.

3.5 Hit/Miss Observability

Each cacheable service method logs a message on a cache miss (the method body only executes on a miss, so this required no additional conditional logic). A custom cache decorator (ObservableCache / ObservableCacheManager) wraps the underlying Redis caches to count hits and misses per cache name, exposed via a custom Spring Boot Actuator endpoint at **/actuator/cachemetrics**:

```
{"restaurants": {"hits": 2, "misses": 1, "hitRatio": 0.667}, "menu": {"hits": 1, "misses": 1, "hitRatio": 0.5}}
```

Note: Spring Boot 4's built-in Micrometer-based cache metrics (cache.gets) were investigated first, but required a separate starter module (spring-boot-starter-micrometer-metrics) not bundled with Actuator by default, and the automatic Redis-to-Micrometer statistics binding did not surface data in this Spring Boot 4.1.1 / Spring Data Redis 4.0 combination at the time of writing. The custom decorator above was built as a reliable, fully self-contained alternative and was verified end-to-end with real requests.

3.6 Automated Tests

19 new unit tests were added covering the custom caching components: hit/miss counting and ratio calculation (CacheMetrics), hit/miss detection and correct delegation of every cache operation (ObservableCache), and cache-wrapping behaviour including null-safety and instance reuse (ObservableCacheManager). Actual Redis interaction (population, invalidation, and resilience) was verified manually against a live Redis instance rather than re-implemented as integration tests, since Spring's caching annotations only take effect within a full application context.

Result: full suite of 52 tests passing, JaCoCo coverage check passed ($\geq 90\%$ line coverage maintained).

4. Performance Verification (k6)

A dedicated load test (50 concurrent virtual users, 30 seconds, repeatedly calling GET /restaurants and GET /menu?restaurantId= against a small, reused set of restaurants) was run twice: once with Redis running, and once with Redis stopped, to directly measure the caching layer's performance impact.

Metric	With Redis (cache active)	Without Redis (DB only)
Average response time	14.4 ms	2.02 s
95th percentile response time	31.0 ms	2.07 s
Throughput	765 requests/sec	22.7 requests/sec
Total requests completed (~30s)	23,063	801
Failed requests	0%	0%

Result: caching reduced average response time by roughly 140x and increased throughput by roughly 34x, with zero failed requests in either configuration.

A notable, honest finding: the "without Redis" average (2.02s) closely matches the configured 2-second Redis connection timeout. With Redis completely unreachable, every request still attempts the Redis connection first, waits out the full timeout, and only then falls back to PostgreSQL. The application therefore remains functionally correct with Redis down (0% failures, confirmed above) but throughput is bounded by that timeout until Redis recovers — a real, documented trade-off rather than a defect.

5. Conclusion

The second milestone is complete. All four issues raised in the first milestone's review have been resolved and verified with evidence, and a complete Redis caching layer has been added to the two most frequently called endpoints, covering cache key design, TTL policy, cache invalidation, graceful degradation when Redis is unavailable, and hit/miss observability. The performance benefit of caching was measured directly and shows an approximately two-order-of-magnitude improvement in response time under sustained concurrent load, while the application was confirmed to remain fully functional, with zero errors, even when the cache layer is entirely unavailable.